



SiFive iommu22_sv48_ats_max User Guide

24G1.02.00

Copyright © 2024 by SiFive, Inc. All rights reserved.

CONFIDENTIAL INFORMATION

SiFive iommu22_sv48_ats_max User Guide

Proprietary Notice

Copyright © 2024 by SiFive, Inc. All rights reserved.

Information in this document is provided “as is,” with all faults.

SiFive expressly disclaims all warranties, representations, and conditions of any kind, whether express or implied, including, but not limited to, the implied warranties or conditions of merchantability, fitness for a particular purpose and non-infringement.

SiFive does not assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation indirect, incidental, special, exemplary, or consequential damages.

SiFive reserves the right to make changes without further notice to any products herein.

Contents

List of Tables	4
List of Figures	5
1 Introduction	6
1.1 About this Document	6
1.2 IOMMU-22 Configuration Parameters	6
1.3 Block Diagram	7
1.4 IOMMU-22 Signal List	10
1.4.1 Clock, Reset, Debug, RAS, Test and Tie-offs	10
1.4.2 PMA Interface	10
1.4.3 Interrupts	12
1.4.4 Host interface	12
1.4.4.1 AHB-Lite Slave interface	13
1.4.5 PDS Interface	14
1.4.5.1 AXI4 Master Interface	15
1.4.6 Device Translation Request Interface	18
1.4.7 Device Translation Completion Interface	21
1.4.8 RAS Interface	24
2 Deliverables	25
2.1 Folder Structure	25
3 Functional Description	27
3.1 Global flow	27
3.2 ATS Translation Request Flow	28
3.3 Page Request Flow	30
3.4 Translated Request Flow	32
3.5 ATS Module	34

3.5.1	DTI Connection Request	34
3.5.2	ATS Translation Request	35
3.5.3	ATS Invalidation Request	38
3.5.4	ATS Page Request	40
3.6	PDS Interface Module	43
3.7	Queue Manager Module	43
3.7.1	Command Queue	43
3.7.2	Fault Queue	45
3.8	Host interface Module	45
3.9	Hardware Performance Monitor Module	46
4	Example Device Tree	47
5	Implementation	49
5.1	Top-Level	49
5.2	Memories	51
5.2.1	RAM Instances	51
5.3	Clock Gates	53
5.4	Simulating with Modified RTL	53
5.5	LINT	53
5.5.1	Directory Structure	54
5.5.2	Running Lint Simulations	54
5.6	CDC	55
5.6.1	Directory Structure	55
5.6.2	Running CDC Simulations	55
5.7	Synthesis	56
5.7.1	SDC File	56
5.7.2	Retiming	57
5.8	DFT	57
5.8.1	DFT Rule Check	57
5.8.2	Directory Structure	57
5.8.3	Running DFT Rule Checks	58
5.9	Gate-Level Simulation	58

A Configuration Options 59

B Top-Level Signals..... 61

C Revision History 64

List of Tables

Table 1 List of IOMMU-22 Parameters for iommu22_sv48_ats_max 7

Table 11 IOMMU SRAM Modules and Configuration 52

Table 12 SRAM Signals..... 52

Table 13 DFT Input Signals..... 57

Table 14 Revision Information 64

List of Figures

Figure 1	IOMMU-22 Block Diagram.....	8
Figure 2	Ready Signal Asserted After ReqVa1	20
Figure 3	Ready Signal Asserted Before ReqVa1	20
Figure 4	Ready and ReqVa1 Asserted Together`	21
Figure 5	Untranslated Request Translation Flow.....	27
Figure 6	ATS Translation Request Flow	29
Figure 7	ATS Page Request Message Flow	31
Figure 8	Translated Request Flow.....	33
Figure 9	DIT-ATS Channel States	34
Figure 10	DIT-ATS Connect and Disconnect Flow	35
Figure 11	Circular Buffer Command Queue Example	44
Figure 12	Example Implementation Flow	50
Figure 13	SRAM Timing	51

Chapter 1

Introduction

1.1 About this Document

This document describes the information necessary to integrate the SiFive IOMMU-22 address translation engine (previously known as PLATEv2) into an SoC. The SiFive IOMMU-22 is compliant to the RISC-V IOMMU Specification (Version 1.0.0, July 2023). This document should be read in conjunction with the RISC-V IOMMU Specification: <https://github.com/riscv-non-isa/riscv-iommu/releases/download/v1.0.0/riscv-iommu.pdf>.

IOMMU-22 also supports DTI-ATSV2 as defined in the Arm® AMBA® Distributed Translation Interface (DTI) Protocol Specification (ARM IHI 0088E.b).

1.2 IOMMU-22 Configuration Parameters

Table 1 shows IOMMU-22 parameters for the iommu22_sv48_atl_max configuration.

Parameter	Value	Definition
DIDWidth	16	Width of the DeviceID
PIDWidth	20	Width of the ProcessID
PSCIDWidth	20	Width of PSCID
GSCIDWidth	16	Width of GSCID
DID_nEntries	128	Number of entries in DeviceID Directory Cache
PID_nEntries	128	Number of entries in ProcessID Directory Cache
L1TLB_nEntries	64	Number of entries in L1TLB
L2TLB_nEntries	4096	Number of entries in L2TLB
L2TLB_nWays	8	Number of ways in L2TLB
Parallel_PTW	8	Number of Parallel PTW modules
L1C_nEntries	32	Number of entries in Level 1 PTE Cache
L2C_nEntries	64	Number of entries in Level 2 PTE Cache
L3C_nEntries	128	Number of entries in Level 3 PTE Cache
gLeaf_nEntries	256	Number of entries in G-stage leaf PTE Cache in nested translation
ReqIDWidth	8	Width of the device translation request ID which represents the amount of outstanding translation requests (sum of read and write outstanding device capability) supported by the devices.
VAS	50	Virtual Address Size
PAS	48	Physical Address Size
ATS	1	ATS support
SvN	Sv48	Maximum supported RISC-V virtual memory system

Table 1: List of IOMMU-22 Parameters for iommu22_sv48_atc_max

1.3 Block Diagram

IOMMU-22 is located in the SoC close to the devices that it provides address translation for. Its role is to intercept the device address transaction to system memory and perform the required address translation, thanks to its TLBs (Translation Lookaside Buffers). If the TLBs don't have the requested information, the IOMMU-22 performs the page table walk through its PDS interface. It manages the Command queue, Fault queue and Page Request queue (if present) to interface with software. Finally, IOMMU-22 can optionally implement an Address Translation Services interface following the PCIe protocol.

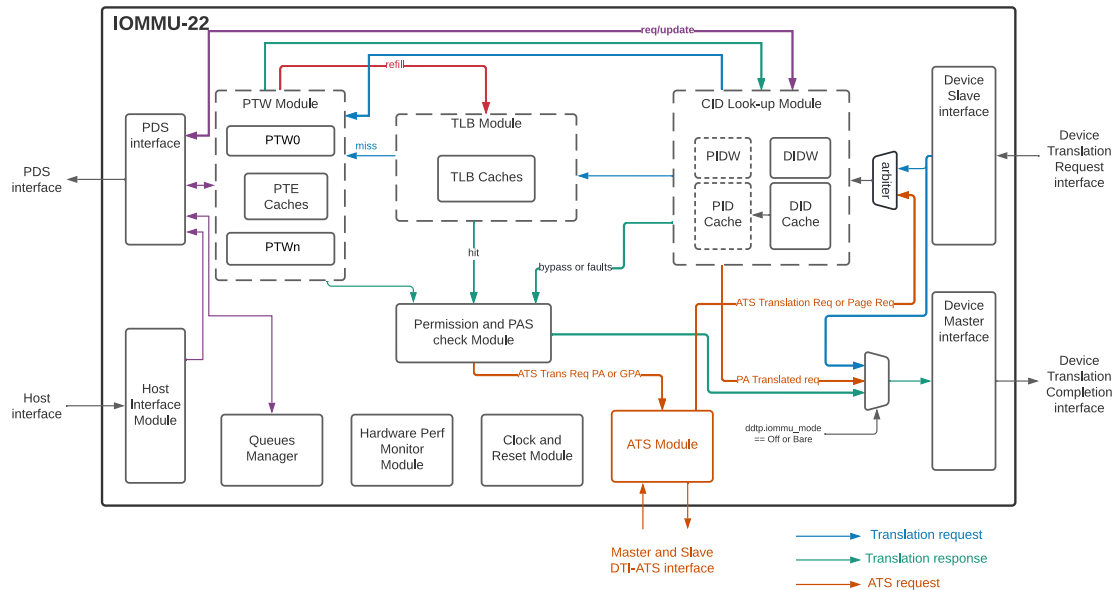


Figure 1: IOMMU-22 Block Diagram

Device Slave Interface: This module manages the Device Translation Request interface which is a valid/ready type of interface.

CID Look-Up Module: This module performs the DeviceID Directory Table look up with the DeviceID and potentially the ProcessID Directory table look up with the ProcessID in order to provide the context information along with the translation request to the TLB module. The DeviceID or ProcessID Directory Tables are located in system memory. This module interacts with the PDS interface to walk the Directory tables and also with the PTW in order to translate pointers that may be GPAs.

TLB Module: Translation Lookaside Buffer Module includes caches. On a TLB hit, the translated address and permissions are sent to a hit queue.

PTW Module: Page Table Walk module. This module implements parallel Page Table Walker(s) (up to Parallel_PTW instances) which share a cache for non-leaf Page Tables.

PDS interface: PLATE Data Structure master interface to the system interconnect in order to access Page Tables, queues and Directory tables stored in system memory (AXI4 master).

Permission Check Module: This module includes an arbiter to get translated requests. It performs the permission checks and sends responses to the Device Master interface. In the case of a violation, it also writes an Error Record into the appropriate Fault Queue via the Queue Manager.

Device Master Interface: This module manages the Device Translation Completion interface which is a valid/ready type of interface.

Host Interface: Slave interface to the system interconnect (AHB Lite slave). This module holds all IOMMU-22 configuration registers.

Queues Manager: This module manages the Command Queue, Fault Queue and optionally the Page Request Queue located in system memory.

Hardware Performance Monitor Module: This module counts performance-related events.

Clock and Reset Module This module implements clocking and reset.

ATS Module: The ATS module interfaces with the CID Module to perform ATS Translation Request for the endpoint. The result of the translation request is provided by the PTW Module. The ATS module also interfaces with the Queues Manager to potentially generate a Fault Record into the Fault Queue, to transmit a Page Request Record to the Page Request Queue or to send an ATS invalidation request to the endpoint. The ATS module interfaces with the Root Port via AXI4 Master and Slave ports.

1.4 IOMMU-22 Signal List

1.4.1 Clock, Reset, Debug, RAS, Test and Tie-offs

Signal Name	Verilog Name	Width	I/O	Description
clock	clock	1	I	Clock signal. Synchronous signals are sampled on the rising edge of the clock
reset	reset	1	I	Reset signal, active high. Asynchronous assert, synchronous deassert with respect to rising edge of clock
psd_test_clock_enable	psd_test_clock_enable	1	I	Enable test clock - bypass gated clock
dbg_disable	dbg_disable	1	I	To prevent usage of the debug registers
Source_CXL	source_cxl	1	I	Indicates that the Root Port connected to the DTI-ATS interface is a CXL port (only if ATS is present)
eb_error_id	eb_error_id	1	O	RAS Interrupt: at least one error reported by the Error Bank in IOMMU-22
iommu_mode	iommu_mode	1	I	For selecting Off(0) mode or Bare(1) mode at startup.

Table 2: Clock, Reset, Debug, RAS, Test and Tie-offs

1.4.2 PMA Interface

This interface is used by the ATS module when `Source_CXL` is set in order to retrieve the memory type of the PA range when the page table indicates PMA. This interface is present if ATS capability is supported.

Signal name	Verilog Name	Width	I/O	Description
pma_lookup_req_bits_size	pma_lookup_req_bits_pa	PAS-12 (1)	O	Physical Address for the memory type request
pma_lookup_req_bits_size	pma_lookup_req_bits_size	4	O	Page Size 4'b0000: 4KiB page 4'b0010: 64KiB page 4'b0011: 2MiB page 4'b0110: 1GiB page All other values are reserved.
pma_lookup_req_valid	pma_lookup_req_valid	1	O	Request Valid
pma_lookup_req_ready	pma_lookup_req_ready	1	I	IO bridge is ready to accept the PMA request
pma_lookup_resp_bits_pma	pma_lookup_resp_bits_pma	2	I	Physical Memory Attributes specific to the platform: 0: Error (unmapped region or conflicting memory type) 1: NC, non-cacheable main memory (RVWMO) or IO, non-cacheable strongly ordered IO 2: C, cacheable main memory 3: Reserved
pma_lookup_resp_valid	pma_lookup_resp_valid	1	I	PMA Response valid.
pma_lookup_resp_valid	pma_lookup_resp_ready	1	O	IOMMU's ATS module is ready

Signal name	Verilog Name	Width	I/O	Description
				to accept the PMA Response.

⁽¹⁾ **PAS** is the Physical Address Size (set to 48 bits).

1.4.3 Interrupts

IOMMU-22 provides one interrupt line:

Signal name	Verilog Name	Width	I/O	Description
INT	wint	1	O	Wired interrupt line (level-sensitive)

1.4.4 Host interface

The Host Interface is a slave interface to IOMMU-22 to access its internal register space for configuration or maintenance:

1.4.4.1 AHB-Lite Slave interface

Signal name	Verilog Name	Width	I/O	Description
AHB-Lite Slave Signals				
Host_HSEL	host_ahb_0_hsel	1	I	Indicates that the current transfer is intended for IOMMU-22 Host port. When it is selected, it must also monitor the status of Host_HREADY to ensure that the previous bus transfer has been completed, before it responds to the current transfer.
Host_HADDR	host_ahb_0_haddr	13	I	Address of the read or write transaction
Host_HBURST	host_ahb_0_hburst	3	I	Indicates if the transfer is a single transfer or forms part of a burst
Host_HMASTLOCK	host_ahb_0_hmastlock	1	I	When HIGH, indicates that the current transfer is part of a locked sequence
Host_HPROT	host_ahb_0_hprot	4	I	Indicates the protection control signals
Host_HSIZE	host_ahb_0_hsize	3	I	Size of a data transfer
Host_HTRANS	host_ahb_0_htrans	2	I	Indicates the transfer type of the current transfer
Host_HWDATA	host_ahb_0_hwdata	64	I	Write data
Host_HWRITE	host_ahb_0_hwrite	1	I	When HIGH, indicates a write transfer and when LOW a read transfer
Host_HRDATA	host_ahb_0_hrdata	64	O	Read data
Host_HREADYOUT	host_ahb_0_hreadyout	1	O	When HIGH, indicates that the transfer has finished on the bus
Host_HRESP	host_ahb_0_hresp	1	O	Indicates the status of the read transfer When LOW, indicates that the transfer status is OKAY When HIGH, indicates that the transfer status is ERROR
Host_HREADY	host_ahb_0_hready	1	I	Indicates to the master and all slaves that the previous transfer is complete

Table 3: Host Interface

Signal Name	Verilog Name	Description
H*USER	N/A	User-defined extension

Table 4: Unused or Unimplemented Signals

1.4.5 PDS Interface

The PDS (PLATE Data Structure) Interface is the IOMMU-22 AXI4 master interface to the system interconnect used to perform implicit accesses to data structures (context table, page tables and queues) located in the system memory. This port is also used to generate MSIs when enabled.

1.4.5.1 AXI4 Master Interface

Signal Name	Verilog Name	Width	I/O	Description
Read Address Channel Signals				
PDS_ARReady	pds_axi4_0_ar_ready	1	I	Indicates that the slave is ready to accept an address and control signals
PDS_ARValid	pds_axi4_0_ar_valid	1	O	Indicates that valid read address and control information are available
PDS_ARID	pds_axi4_0_ar_bits_id	4	O	Identification tag for the read transaction
PDS_ARAddr	pds_axi4_0_ar_bits_addr	PAS ⁽¹⁾	O	Address of the first transfer in a read transaction
PDS_ARLen	pds_axi4_0_ar_bits_len	8	O	Length: Indicates the number of transfers in a burst
PDS_ARSize	pds_axi4_0_ar_bits_size	3	O	Size of each transfer in a burst
PDS_ARBurst	pds_axi4_0_ar_bits_burst	2	O	Burst type
Read Data Channel Signals				
PDS_RReady	pds_axi4_0_r_ready	1	O	Indicates that the master is ready to accept the read data
PDS_RValid	pds_axi4_0_r_valid	1	I	Indicates that valid read data and strobes are available
PDS_RData	pds_axi4_0_r_bits_data	64	I	Read data
PDS_RID	pds_axi4_0_r_bits_id	4	I	ID tag of the read data transfer which must match the ARID value of the read transaction
PDS_RResp	pds_axi4_0_r_bits_resp	2	I	Indicates the status of the read transfer
PDS_RLast	pds_axi4_0_r_bits_last	1	I	Indicates the last transfer in a read burst
Write Address Channel Signals				
AWReady	pds_axi4_0_aw_ready	1	I	Indicates that the slave is ready to accept an address and control signals
AWValid	pds_axi4_0_aw_valid	1	O	Indicates that valid write address and control information are available
AWID	pds_axi4_0_aw_bits_id	4	O	Identification tag for the write transaction

Table 5: PDS interface signals

Signal Name	Verilog Name	Width	I/O	Description
AWAddr	pds_axi4_0_aw_bits_addr	PAS ⁽¹⁾	O	Address of the first transfer in a write transaction
AWLen	pds_axi4_0_aw_bits_len	8	O	Length: Indicates the number of transfers in a burst
AWSize	pds_axi4_0_aw_bits_size	3	O	Size of each transfer in a burst
AWBurst	pds_axi4_0_aw_bits_burst	2	O	Burst type
Write Data Channel Signals				
WReady	pds_axi4_0_w_ready	1	I	Indicates that the slave is ready to accept the written data
WValid	pds_axi4_0_w_valid	1	O	Indicates that valid write data and strobes are available
WData	pds_axi4_0_w_bits_data	64	O	Write data
WStrb	pds_axi4_0_w_bits_strb	8	O	Write strobe
WLast	pds_axi4_0_w_bits_last	1	O	Indicates the last transfer in a write burst
BReady	pds_axi4_0_b_ready	1	O	Indicates that the master can accept the response information
BValid	pds_axi4_0_b_valid	1	I	Indicates that a valid write response is available
BID	pds_axi4_0_b_bits_id	4	I	Identification tag of the write response which matches the AWID tag of the write transaction to which the slave is responding
BResp	pds_axi4_0_b_bits_resp	2	I	Indicates the status of the write transaction

Table 5: PDS interface signals

⁽¹⁾ **PAS** is the Physical Address Size (set to 48 bits)

Signal Name	Verilog Name	Description
ARLOCK	pds_axi4_0_ar_bits_lock	Indicates the lock type used in an atomic transfer
ARCACHE	pds_axi4_0_ar_bits_cache	Cache attributes of a transaction
ARQOS	pds_axi4_0_ar_bits_qos	Quality of service signaling
ARPROT	pds_axi4_0_ar_bits_prot	This signal indicates protection level and transaction access type
ARREGION	N/A	Region indicator for the transaction
ARUSER	N/A	User-defined extension for the address channel
AWLOCK	pds_axi4_0_aw_bits_lock	Indicates the lock type used in an atomic transfer
AWCACHE	pds_axi4_0_aw_bits_cache	Cache attributes of a transaction.
AWPROT	pds_axi4_0_aw_bits_prot	This signal indicates protection level and transaction access type
AWQOS	pds_axi4_0_aw_bits_qos	Quality of service signaling
AWREGION	N/A	Region indicator for the transaction
AWUSER	N/A	User-defined extension for the address channel
WUSER	N/A	User-defined extension for the write data channel
BUSER	N/A	User-defined extension for the write response channel

Table 6: Unused or Unimplemented Signals

Note that the PDS Master Port can only generate the INCR burst type, and not WRAP or FIXED.

1.4.6 Device Translation Request Interface

The DTRI (Device Translation Request interface) is an IOMMU-22 slave interface, which receives the IO virtual address translation requests from one or more IO devices:

Signal Name	Verilog Name	Width	I/O	Description
DID	dtri_bits_did	DIDWidth	I	DeviceID
PID	dtri_bits_pid	PIDWidth	I	ProcessID (present if PIDWidth is not 0)
PID_VALID	dtri_bits_pid_valid	1 bit	I	ProcessID Valid (present if PIDWidth is not 0)
VAddr	dtri_bits_va	VAS-12 ⁽²⁾	I	Device virtual address
ReqVal	dtri_valid	1	I	Request valid signal
ReqID	dtri_bits_reqid	ReqIDWidth ⁽³⁾	I	Request Identification Number
RWn	dtri_bits_rwn	1	I	Read when high, write when low
TRANSLATED	dtri_bits_translated	1	I	Translated (1) or Untranslated (0). (Always 0 if ATS is not supported)
Exe	dtri_bits_exe	1	I	Execution mode (present if PIDWidth is not 0)
Priv	dtri_bits_priv	1	I	Privilege mode (1: Supervisor, 0: User). (present if PIDWidth is not 0)
Ready	dtri_ready	1	O	Ready signal to indicate back pressure to the device

Table 7: Device Translation Request Interface Signals

⁽²⁾ **VAS** is the Virtual Address Size (set to 50 bits)

⁽³⁾ **ReqIDWidth** is the width of the ReqID field (set to 6 bits for small version and 8 bits for others)

The lower 12 bits of the device virtual address must be stored in an IO bridge for instance (along with the data) in order to append to the physical address provided on the device translation completion interface and construct the final physical address.

This device interface uses a valid/ready handshake protocol similar to AXI. The device (the source) generates the ReqVal signal to indicate when the address and control information are available. IOMMU-22 (the destination) generates the Ready signal to indicate that it can accept the transaction. Transfer occurs only when both the ReqVal and Ready signals are HIGH.

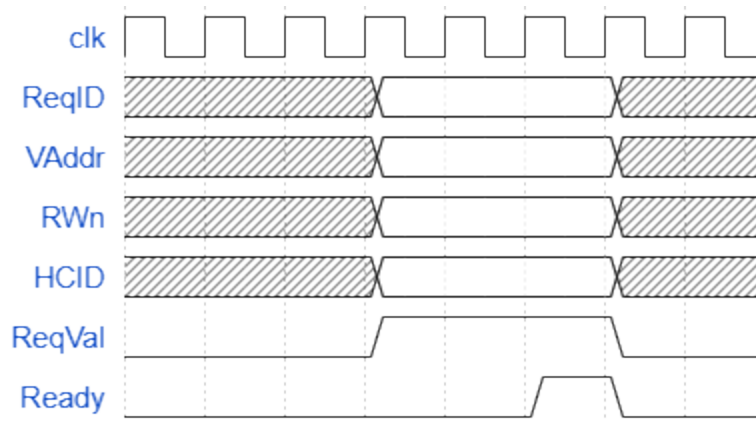


Figure 2: Ready Signal Asserted After ReqVal1

In the above waveform, the device presents information and asserts the ReqVal1 signal while IOMMU-22 cannot accept the request. The device must keep the ReqVal1 and the information asserted and stable until IOMMU-22 asserts the Ready signal. The device is not permitted to wait until Ready is asserted before asserting ReqVal1. When ReqVal1 is asserted, it must remain asserted until the handshake occurs, at a rising clock edge when ReqVal1 and Ready are both asserted.

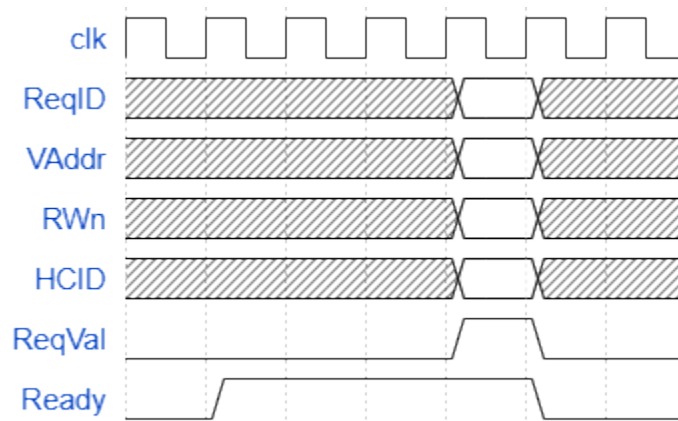


Figure 3: Ready Signal Asserted Before ReqVal1

IOMMU-22 asserts Ready before the device transaction is valid. This assertion indicates that it can accept the information. Then the device presents the information and asserts ReqVal. In this case, the device translation request occurs in a single cycle.

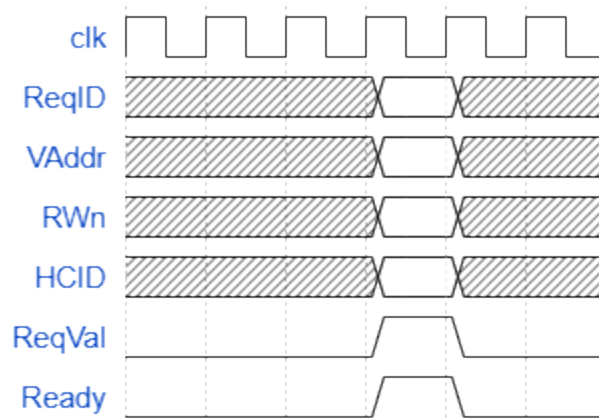


Figure 4: Ready and ReqVal1 Asserted Together

Both the device and IOMMU-22 indicate that they can transfer the translation request. In this case, the transaction occurs at the rising clock edge when the assertion of both ReqVal1 and Ready can be recognized. Also in this case, the device translation request occurs in a single cycle.

The Request Identification Number must be unique for each in-flight transaction. When the IO Bridge makes a translation request to IOMMU-22, it must assign a ReqID along with ReqVal1. When IOMMU-22 completes the translation, it indicates the result on the device Translation Completion interface and provides the CompID to the device. This identification number can then be reused on the Device Translation Request interface for a new translation request.

1.4.7 Device Translation Completion Interface

The DTCL (Device Translation Completion Interface) is an IOMMU-22 master port that provides the requested translated Physical Address and an error status:

Signal Name	Verilog Name	Width	I/O	Description
PAddr	dtci_bits_pa	PAS-12 ⁽¹⁾	O	Device Physical Address
PBMT	dtci_bits_pbmt	2	O	Page Based Memory Type (0: PMA, 1: NC, 2: IO, 3: reserved) (Always set to 0 if Svpbmt is not supported)
CompVal	dtci_valid	1	O	Completion Valid signal
CompID	dtci_bits_reqid	ReqIDWidth ⁽³⁾	O	Completion Identification Number corresponding to the DTRI ReqID
ErrFlag	dtci_bits_error_flag	1	O	Error Flag Indication
Ready	dtci_ready	1	I	Ready signal to indicate back pressure from the device
DrainRd	drain_rd	1	O	Drain Rd channel
DrainRdAck	drain_rd_ack	1	I	Drain Rd channel Acknowledge, that it guarantees that previous reads have been completed
DrainWr	drain_wr	1	O	Drain Wr channel
DrainWrAck	drain_wr_ack	1	I	Drain Wr channel Acknowledge, that it guarantees that previous writes have been completed

Table 8: Device Translation Completion Interface Signals

⁽¹⁾ **PAS** is the Physical Address Size (set to 48 bits)

⁽³⁾ **ReqIDWidth** is the width of the ReqID field (set to 6 bits for small version and 8 bits for others)

The Device Translation Completion Interface follows the same protocol as described above.

The AXI4-Stream link for DTI consists of two AXI4-Stream interfaces, one for each direction. The mapping of AXI4-Stream signals to the DTI protocol is described in the Arm AMBA Distributed Translation Interface (DTI) Protocol Specification, Edition 3.

DTIS_dn (downstream) (present if ATS capability is supported)

- Receives Translation Requests
- Receives Invalidation Completion

Signal Name	Verilog Name	Width	I/O	Description
DTIS_dn_tvalid	dtis_dn_valid	1	I	Indicates that the master is driving a valid transfer. A transfer takes place when both DTIS_dn_tvalid and DTIS_dn_tready are asserted.
DTIS_dn_tready	dtis_dn_ready	1	O	Indicates that the slave DTIS_dn port can accept a transfer in the current cycle.
DTIS_dn_tdata	dtis_dn_bits_data	256	I	Data
DTIS_dn_tkeep	dtis_dn_bits_keep	32	I	Indicates which bytes of DTIS_dn_tdata contains valid data.
DTIS_dn_tlast	dtis_dn_bits_dest	1	I	Indicates the last beat of the transfer

DTIS_dn_tid is not implemented, as only one Root Port with ATS capability can be managed by IOMMU-22.

DTIM_up (upstream) (present if ATS capability is supported)

Signal Name	Verilog Name	Width	I/O	Description
DTIM_up_tvalid	dtim_up_valid	1	O	Indicates that the master is driving a valid transfer. A transfer takes place when both DTIM_up_tvalid and DTIM_up_tready are asserted.
DTIM_up_tready	dtim_up_ready	1	I	Indicates that the slave DTIM_up port can accept a transfer in the current cycle.
DTIM_up_tdata	dtim_up_bits_data	256	O	Data
DTIM_up_tkeep	dtim_up_bits_keep	32	O	Indicates which bytes of DTIM_up_tdata contains valid data.
DTIM_up_tlast	dtim_up_bits_dest	1	O	Indicates the last beat of the transfer

1.4.8 RAS Interface

Signal Name	Verilog Name	Width	I/O	Description
eb_error_id	eb_error_id	1	O	RAS interrupt: At least one Error ID is non-zero from all of the status registers in this error bank.
eb_reset	eb_reset	1	I	Error bank reset signal: - active high - asynchronous assert - synchronous deassert with respect to rising edge of clock

Table 9: RAS Signals

Note

The Error Bank registers implemented at `IOMMU_BASE + 0x1000` are only reset by the `eb_reset` signal.

Chapter 2

Deliverables

This chapter describes the contents of the IP deliverables for the iommu22_sv48_ats_max.

2.1 Folder Structure

The folder structure of the delivery is as follows:

release-notes.txt

Contains important information about the release deliverables.

docs

Contains the following documents and directories:

- iommu22_sv48_ats_max Manual and User Guide PDF files specific to this configuration
- systemip_configuration.txt, which summarizes the configuration options selected
- doc/uvm_env.adoc, which describes the IOMMU demo testbench

info

Files that describe the design.

ipxact-*.xml

IP-XACT IEEE 1685-2009 documents containing descriptions of the memory-mapped registers, ports, and bus interfaces within the IP. The IP-XACT file *will not* describe any non-memory-mapped control and status registers (CSRs).

mems.conf

Configuration file that describes the memory instances of the design.

sifive_scope.yml

Contains a .yml description of the SiFive Scope signals included in the design.

rtl

The RTL.

design

SystemVerilog of the IOMMU.

implementation

Contains files needed for synthesis and to run Lint, CDC, RDC, and DFT checks.

cdc

Contains scripts to run CDC checks.

dft

Contains scripts to run DFT checks.

lint

Contains scripts to run Lint checks.

rdc

Contains Reset Domain Crossing checks.

synth

Contains an SDC file for timing constraints and a list of modules that need to be retimed.

memories

A single Verilog file containing the behavioral simulation models of all memories in the design and testbench.

sram_integration_tb

Contains a standalone testbench for SRAM integration.

testbench

Includes all the modules in the simulation testbench, the test driver, and extracted simulation constructs (assertions) that are bound to locations in the DUT.

Makefile

Used to execute the testbench.

Chapter 3

Functional Description

3.1 Global flow

The following diagram describes the global flow of VA-PA address translation through IOMMU.

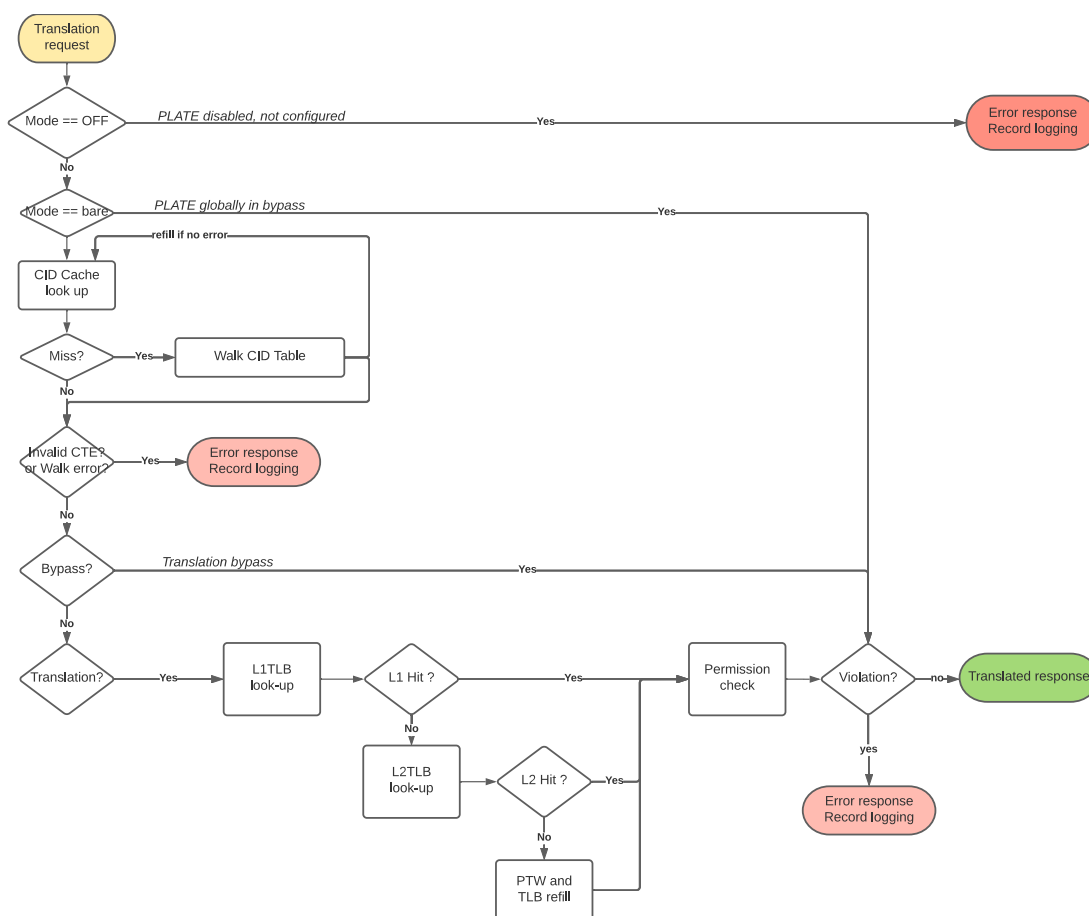


Figure 5: Untranslated Request Translation Flow

IOMMU-22 receives a translation request on its Device Translation Request interface with a DeviceID and optionally a ProcessID.

1. If IOMMU-22 is not yet initialized, `ddtp.iommu_mode` is set to OFF which indicates that IOMMU-22 is disabled and that all translation requests must be aborted (responded with an error).
2. If IOMMU-22 is configured to globally bypass via `ddtp.iommu_mode` set to BARE, IOMMU-22 will not perform any address translation nor memory protection.
3. If IOMMU-22 is set to perform address translation (by setting `ddtp.iommu_mode` to 1LVL or 2LVL), it starts by performing the ContextID look up using the DeviceID in the Device Directory Cache. Since this table may be a multi level table stored in system memory, multiple reads via the PDS master port may be required (in case of miss).
 - a. An invalid ContextID Entry (CTE) is found or a hardware error occurred. An Error Record is logged and an error response is sent on the completion interface.
 - b. A valid CTE is found, IOMMU-22 can continue its process to step 4.
4. If no translation is requested in the CTE (`iosatp.Mode` and `iohgatp.Mode` are set to bare), IOMMU-22 does not perform any address translation on this request nor protection.
5. If single stage, nested or G-stage only translation is requested, IOMMU-22 performs a first look-up in its L1TLB.
 - a. If it results in a hit, IOMMU-22 proceeds with step 6.
 - b. If it results in a miss, IOMMU-22 performs a second look-up in its L2TLB.
 - i. If it results in a hit, IOMMU-22 proceeds with step 6.
 - ii. If it results in a miss, IOMMU-22 needs to walk the page thanks to the pointer to the base address (PPN) of the page translation table contained in the CTE. The page table is walked using the proper paging scheme for the requested translation (single stage, G-stage only, or nested). Before sending any request to system memory, IOMMU-22 will first look in its local cache for any available matching PTE. If an error occurs at any stage of PTE access, a fault is raised and logged as an Error Record.
6. IOMMU-22 performs all required permission checks. The permission rules are provided by the PTE or TLB. In case of a violation, an Error Record is logged optionally.
7. IOMMU-22 finally sends the translation completion on the Device Translation Completion interface with an error indication, if any.

3.2 ATS Translation Request Flow

The following diagram describes the flow for an ATS translation request through IOMMU:

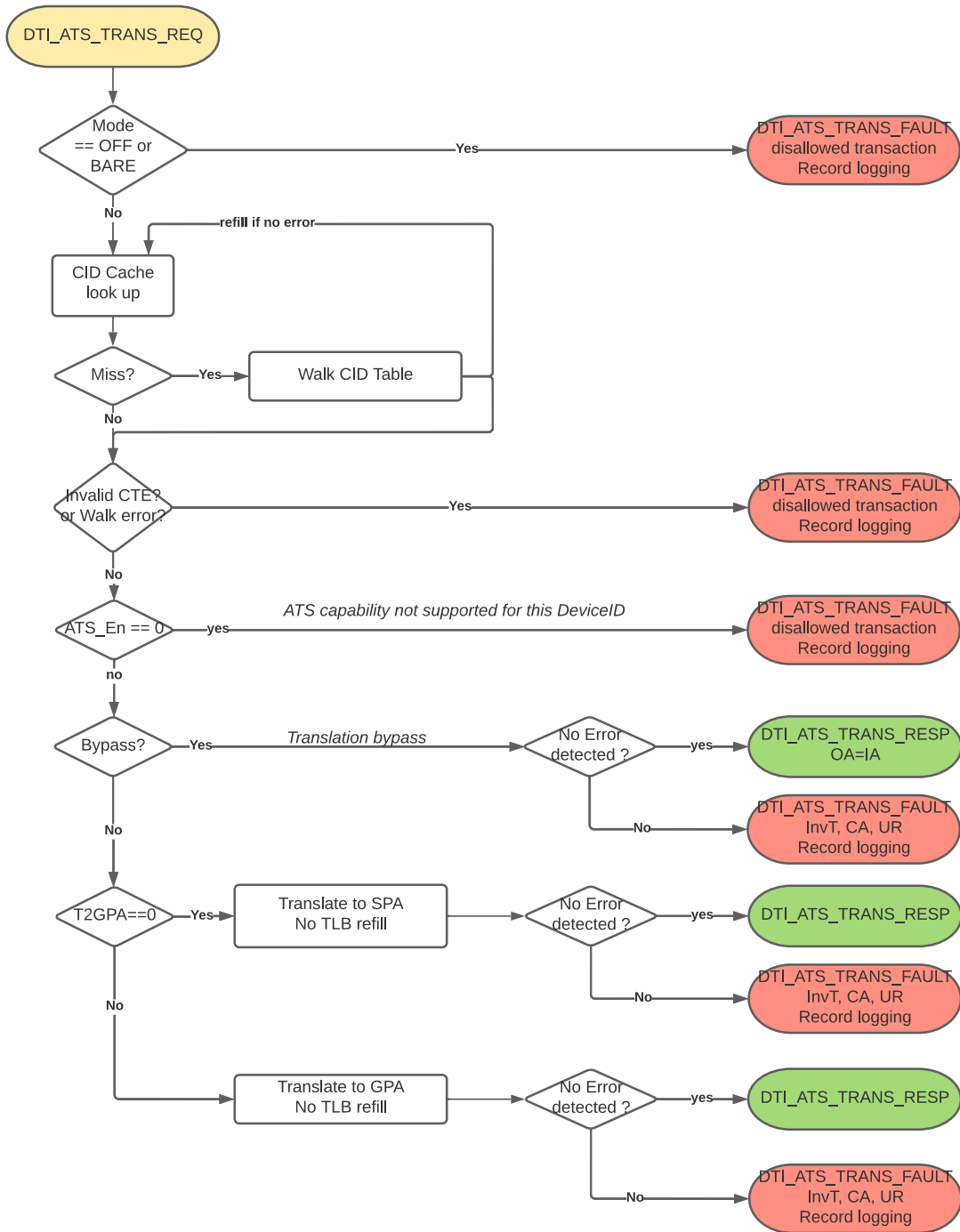


Figure 6: ATS Translation Request Flow

ATS Translation Request messages are received on the DTI Slave (dtis_dn port).

1. If `ddtp.iommu_mode` is bare or off, ATS messages are not allowed to be serviced.
2. IOMMU-22 must retrieve the context information for the DeviceID (and optionally ProcessID).

3. If `ATS_en` is set, the ATS Translation Request can be serviced.
4. If `T2GPA` is set, IOMMU-22 must respond with a GPA as translated address instead of SPA.
5. Finally, IOMMU-22 must generate either a `DTI_ATS_TRANS_RESP` or `DTI_ATS_TRANS_FAULT` on its DTI Master port (`dtim_up`).

3.3 Page Request Flow

The following diagram describes the flow for an ATS Page Request message through IOMMU:

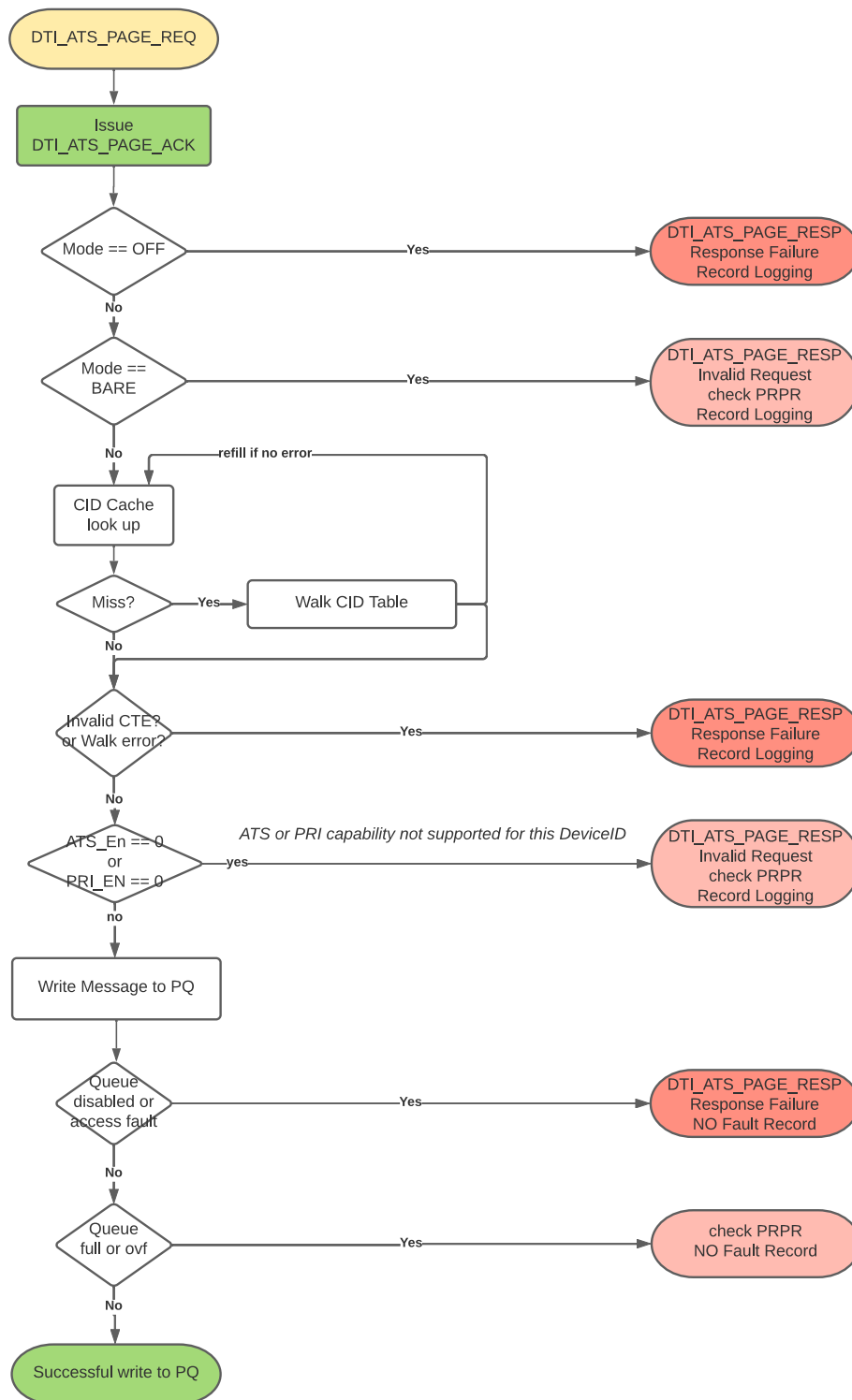


Figure 7: ATS Page Request Message Flow

1. The IOMMU must first acknowledge the PR message.

2. If `ddtp.iommu_mode` is bare or off, ATS messages are not allowed to be serviced. The IOMMU must respond with a response failure to the Root Port (on `dtim_up`).
3. IOMMU-22 must retrieve the context information for the DeviceID (and optionally ProcessID).
4. If `ATS_en` and `PRI_en` are set, the ATS Page Request Message can be serviced and thus forwarded to the PQ.

3.4 Translated Request Flow

The following diagram describes the flow for a translated request through IOMMU:

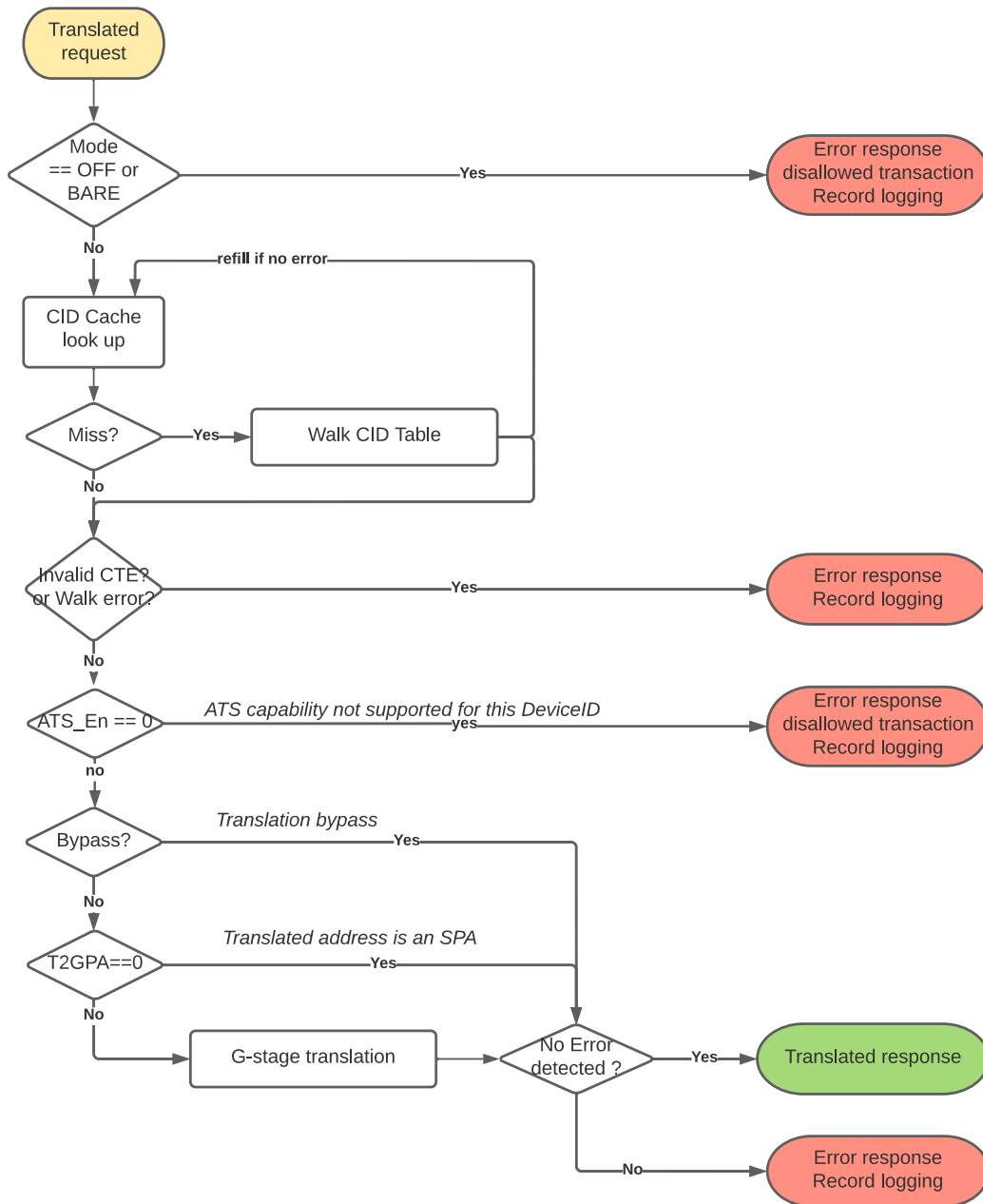


Figure 8: Translated Request Flow

IOMMU-22 may receive a translated request on the `dtr i` port.

1. If `ddtp.iommu_mode` is `bare` or `off`, translated requests are not allowed.
2. IOMMU-22 must retrieve the context information for the DeviceID (and optionally ProcessID).
3. If `ATS_en` is not set, translated requests are not allowed.

4. If `ATS_en` is set and `T2GPA` is set, the address of the translated request is a GPA and must be translated into a SPA.

3.5 ATS Module

The ATS Module must manage the master and slave ports of the DTI-ATS interface.

3.5.1 DTI Connection Request

Out of reset, the DTI-ATS channel state is DISCONNECTED.

The following diagram represents the 4 possible states of a DTI channel.

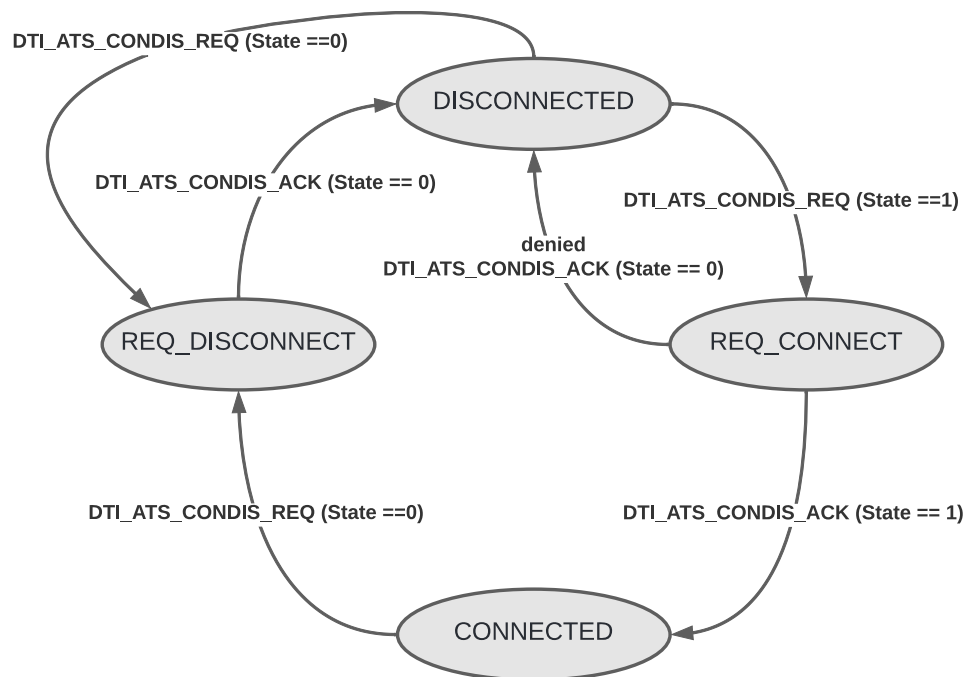


Figure 9: DIT-ATS Channel States

When the channel is in the DISCONNECTED state:

- No message is allowed to be sent to the PCIe RP.

When the channel is in the REQ_CONNECT state:

- The IOMMU can accept or deny the connection request. If it accepts, the IOMMU updates its internal tokens values with the `DTI_ATS_CONDIS_REQ.TOK_TRANS_REQ` if less than 32

and DTI_ATS_CONDIS_REQ.TOK_INV_GNT. IOMMU-22 can support up to 32 outstanding ATS translation requests.

When the channel is in the CONNECTED state:

- Any ATS messages are accepted and managed by the ATS module. Tokens are updated.

When the channel is in the REQ_DISCONNECT state:

- Any outstanding invalidation or synchronization responses are not returned. All invalidation requests are considered to be completed when the PCIe RP enters DISCONNECTED state and invalidates its caches.
- Outstanding DTI_ATS_PAGE_RESPACK messages are not returned.
- The PCIe RP continues to accept protocol appropriate requests from the IOMMU. No response is given to the requests and they can be ignored.

The following diagram describes the flow upon reception of a DTI_ATS_CONDIS_REQ:

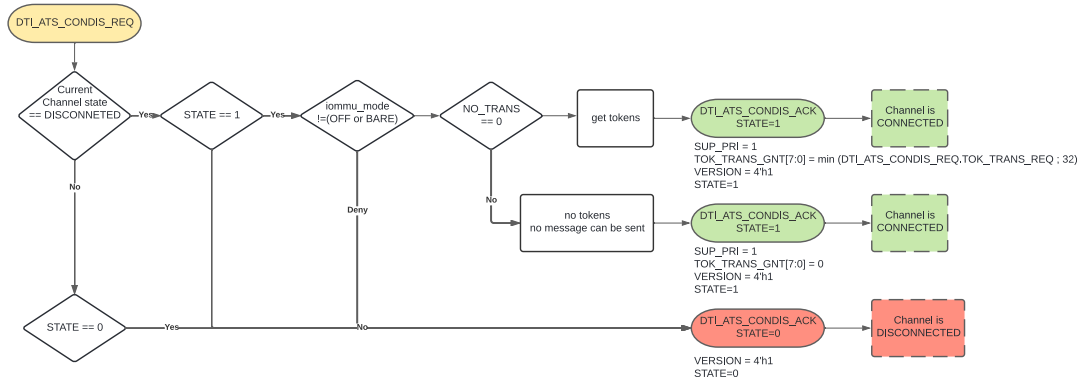


Figure 10: DIT-ATS Connect and Disconnect Flow

3.5.2 ATS Translation Request

An ATS translation request is received on the dtis_dn (DTI_ATS_TRANS_REQ message) with the following parameters:

- IA[63:12]: Input Address which is the IOVA
- SID[31:0]: StreamID which corresponds to the DeviceID
- ssv: Indicates if the SubStreamID (or ProcessID) is present for this request
- SSID[19:0]: SubStreamID which corresponds to the ProcessID, if ssv is set
- nw: No Write (0: Read and Write access, 1: Read only access)
- InD: Instruction and Data access (0: Data access only, 1: Execute and data access)
- PnU: Privileged or Unprivileged (0: Unprivileged, 1: Privileged)

- Protocol: Must be set to 1 to indicate DTI-ATS
- TRANSLATION_ID[7:0]: Identification number of the translation
- QOS[3:0]: Reserved for now (4'h0)
- MST_MSG_TYPE[3:0]: DTI_ATS_TRANS_REQ (4'h2)

The ATS module performs the following steps upon reception of an ATS Translation Request:

1. if `ddtp.iommu_mode` is off or bare, it is a disallowed transaction (UR).
2. Get the context information for the DeviceID and potentially ProcessID
3. If `ATS_en` is not set, the ATS module must respond with a UR (Translation Fault message to Root Port)
4. If `ATS_en` is set, IOMMU-22 must perform the translation
 - a. If `tc.T2GPA` is set, the ATS Module must only perform the VS-stage translation and respond to the ATS Translation Request with the GPA address.
 - b. If `tc.T2GPA` is not set, the ATS Module must perform the S/VS stage and G-stage if needed as it must respond with the SPA address.
5. A Translation Response message must be sent back to the Root Port

The IOMMU TLB is refilled with translations from ATS requests.

Translation Response Message:

- OA[63:12]: Output translated address
Bits within the range given by the `TRANS_RNG` field must match `DTI_ATS_TRANS_REQ.IA`. For example, if the value of `TRANS_RNG` is 2, then `OA[15:12]` must equal `DTI_ATS_TRANS_REQ.IA[15:12]`.
- AMA[2:0]: ATS Memory Attribute. Reserved (3'b000)
- `TRANS_RNG[3:0]`: this field indicates the aligned range of addresses translation is valid for. Supported range:
 - a. 4'b0000: 4KiB
 - b. 4'b0010: 64KiB
 - c. 4'b0011: 2MiB
 - d. 4'b0110: 1GiB
- `ALLOW_X`: Set if exe and read permission are set in the page table and if `DTI_ATS_TRANS_REQ.InD` and `DTI_ATS_TRANS_REQ.SSV` were set, else 0.
- `ALLOW_W`: Set if write permission is set in the page table (when set, read permission is required) and if `DTI_ATS_TRANS_REQ.nW` was 0.
- `ALLOW_R`: Set if read permission is set in the page table

- BYPASS: 1'b0
- CXL_IO: CXL_IO is set to 0 if Source_CXL is not set. If **Source_CXL** is set, CLK_IO setting depends on the memory type. If the PBMT is NC or IO, then CXL_IO should be set. If the PBMT is PMA, the ATS Module must request the memory type for the OA via the PMA interface. If the PMA response is NC or IO, then CXL_IO must be set. Otherwise, CXL_IO is 0. If the PMA response is Error, a DTI_ATS_TRANS_FAULT must be generated with a CA fault type.
- UNTRANSLATED: 1'b0
- TRANSLATION_ID[7:0]: Indicates the identification number of the translation request
- SLV_MSG_TYPE[3:0]: DTI_ATS_TRANS_RESP (4'h2)

Note

If DTI_ATS_TRANS_REQ.PnU == 0, the request does not target privileged memory. So, if the U-bit in the page table is not set (0, not user mode), a success response must be given with ALLOW_R and ALLOW_W both set to 0. A DTI_ATS_TRANS_FAULT with Invalid Translation can be generated.

Note

If DTI_ATS_TRANS_REQ.PnU == 1, the request targets privileged memory. So, if the U-bit in the page table is set (1, user mode) and the ta.SUM == 0, a success response must be given with ALLOW_R and ALLOW_W both set to 0. A DTI_ATS_TRANS_FAULT with Invalid Translation can be generated.

Mapping to PCIe Translation Completion Data entry: .PCIe Translation Completion Data

Field	Mapping
Translated address	OA potentially modified per TRANS_RNG
S	Set if TRANS_RNG != 4'h0
N (Non-snooped access)	1'b0
CXL.io	DTI_ATS_TRANS_RESP.CXL_IO
AMA	3'b000
Global	1'b0
Priv	DTI_ATS_TRANS_REQ.PnU
Exe	DTI_ATS_TRANS_RESP.ALLOW_X
U	DTI_ATS_TRANS_RESP.UNTRANSLATED
W	DTI_ATS_TRANS_RESP.ALLOW_W
R	DTI_ATS_TRANS_RESP.ALLOW_R

When an error is encountered on the ATS translation request execution, a DTI_ATS_TRANS_FAULT is generated on DTIM_up port.

- **FAULT_TYPE:**
 - 2'b00: Invalid Translation (Success with Read and Write bits clear)
 - 2'b01: Completer Abort (CA)
 - 2'b10: Unsupported Request (UR)
 - 2'b11: Reserved
- **TRANSLATION_ID:** Indicates the identification number of the translation request
- **SLV_MSG_TYPE[3:0]:** DTI_ATS_TRANS_FAULT (4'h1)

Mapping error cause to FAULT_TYPE in DTI_ATS_TRANS_FAULT message:

FAULT_TYPE	Error Cause
CA	Instruction access fault (cause = 1) Read access fault (cause = 5) Write/AMO access fault (cause = 7) PDT entry load access fault (cause = 265) PDT entry misconfigured (cause = 267)
UR	All inbound transactions disallowed (cause = 256) DDT entry load access fault (cause = 257) DDT entry not valid (cause = 258) DDT entry misconfigured (cause = 259) Transaction type disallowed (cause = 260)
Invalid Translation	Instruction page fault (cause = 12) Read page fault (cause = 13) Write/AMO page fault (cause = 15) Instruction guest page fault (cause = 20) Read guest-page fault (cause = 21) Write/AMO guest-page fault (cause = 23) PDT entry not valid (cause = 266)

Table 10: Error Cause to FAULT_TYPE

3.5.3 ATS Invalidation Request

Upon reception of an ATS invalidation Command, the ATS Module must transmit an ATS Invalidation Request (DTI_ATS_INV_REQ) to the Root Port via the dtim_up interface if it has a valid invalidation token.

Fields in ATS.INVAL command fields:

- Destination Segment (DSEG if DSV set)
- RID

- PID if PV set
- PAYLOAD = message body which consists of the untranslated address, S bit and Global bit.

DTI_ATS_INV_REQ message:

- VA[63:12]: Address to be invalidated extracted from PAYLOAD[63:12]
- RANGE[5:0]: The range is calculated as 2^{RANGE} addresses, in multiples of 4KiB pages. Set to 4KiB (RANGE=0) if S-bit in PAYLOAD[11] is not set. If S-bit in PAYLOAD[11] bit is set, extract the page size from PAYLOAD[63:12]. RANGE is set to the number of 1 +1 counted until the first 0 is found starting from VA's LSB.
Example: VA[63:12] is xxxx ... xxxx xxxx 0111 and S-bit is 1, then RANGE = 4 (64KiB)
Example: VA[63:12] is xxxx ... xxx0 1111 1111 and S-bit is 1, then RANGE = 9 (2MiB)
Example: VA[63:12] is xxxx ... xx01 1111 1111 and S-bit is 1, then RANGE = 10 (4MiB)
- SID: [DSEG; RID], if DSV==0, DSEG[15:0]=4'h0
- SSID: PID if PV is set
- OPERATION[7:0]:
 - 0x31: PV==0, PAYLOAD[0]==0
 - 0x33: PV==1, PAYLOAD[0]==1 (Global)
 - 0x39: PV==1, PAYLOAD[0]==0
- SLV_MSG_TYPE: DTI_ATS_INV_REQ (4'hc)

When the ATS Module transmits the DTI_ATS_INV_REQ message, it must decrement its invalidation token value.

The ATS Module must collect and ACK.

Upon reception of a DTI_ATS_INV_ACK, the ATS module can increment its invalidation token value.

The ATS Module is expected to send a SYNC request to the Root Port on IOFENCE command.

IOMMU-22 can send a SYNC_REQ even if no ATS invalidation request has been sent (but in a connected state).

The ATS Module can send a DTI_ATS_SYNC_REQ if no pending acknowledge of sync request or invalidation request is expected.

Software is expected to send an IOFENCE command. Timer is started at the IOFENCE command.

3.5.4 ATS Page Request

IOMMU-22 receives an ATS Page Request Message from the Root Port on the dtis_dn port.

The ATS Module must transmit this request to software via the PQ.

DTI_ATS_PAGE_REQ message:

- ADDR[63:12]: Requested page address
- PRG_INDEX[8:0]: Page Request Group Index
- SID[31:0]: StreamID which corresponds to the DeviceID
NOTE: SID [31:24] is ignored.
- SSV: Indicates if the SubStreamID (or ProcessID) is present for this request
- SSID[19:0]: SubStreamID which corresponds to the ProcessID, if SSV is set
- LAST: Indicates whether this message is the last request in a page request group
NOTE: The "stop PASID" marker is indicated by SSV=1, READ=WRITE=0
- WRITE: when set, write access is requested
- READ: when set, read access is requested
- INST: when set, execute access is requested
- PRIV: when set, privileged access is requested
- PROTOCOL: Must be set to 1 (DTI-ATS protocol)
- MST_MSG_TYPE: DTI_ATS_PAGE_REQ (4'h8)

If no error detected, the message can be written to the PQ:

- PAYLOAD[63:0] = {DTI_ATS_PAGE_REQ.ADDR[63:12],
DTI_ATS_PAGE_REQ.PRG_INDEX[8:0], DTI_ATS_PAGE_REQ.LAST,
DTI_ATS_PAGE_REQ.WRITE, DTI_ATS_PAGE_REQ.READ}
- X = DTI_ATS_PAGE_REQ.INST
- PRIV = DTI_ATS_PAGE_REQ.PRIV
- RID = DTI_ATS_PAGE_REQ.SID[15:0]
- PV = DTI_ATS_PAGE_REQ.SSV
- PID = DTI_ATS_PAGE_REQ.SSID if SSV is set
- DID = DTI_ATS_PAGE_REQ.SID[23:0]

Software is then interrupted and services the fault. It completes its process by sending an ATS.PRGR command with the following fields:

- PAYLOAD[63:0] = {Destination Device ID [15:0], Response Code [3:0], Reserved[2:0],
Page Request Group Index[8:0], 32'b0}

- RID is the Requester ID (DeviceID) and is equivalent to SID[15:0]
- PID is the PASID which is equivalent to the SubStreamID SSID if SSV is set
- PV is the ProcessID Valid (equivalent to SSV)
- DSEG is the Destination Segment which corresponds to the original SID[23:16]
- DSV = 1 if SID[23:16] was not 0 in the original request

The ATS module constructs a DTI_ATS_PAGE_RESP message for the Root Port (sent on dtim_up):

- RESP[1:0]: Response Code = ATS.PRGR.PAYLOAD[15:12]
 - 2'b00: Response Failure
 - 2'b01: Invalid Request
 - 2'b10: Success
 - 2'b11: Reserved
- PRG_INDEX[8:0]: ATS.PRGR.PAYLOAD[8:0]
- SID[31:0]: set to {8'h0, ATS.PRGR.DSEG, ATS.PRGR.RID}
- SSID[19:0]: ATS.PRGR.PID
- SSV: ATS.PRGR.PV
- SLV_MSG_TYPE: DTI_ATS_PAGE_RESP (4'h8)

When PRI is enabled for a device, the IOMMU may still be unable to report "Page Request" or "Stop Marker" messages through the PQ due to error conditions such as the queue being disabled, queue being full, or the IOMMU encountering access faults when attempting to access queue memory.

If EN_PRI is set to 0, or EN_ATS is set to 0, or if the IOMMU is unable to locate the dc to determine the EN_PRI configuration, or the request could not be queued into PQ then the IOMMU behavior depends on the type of "Page Request".

- If the "Page Request" does not require a response, i.e. the "Last Request in PRG" field of the message is set to 0, then such messages are silently discarded. "Stop Marker" messages do not require a response and are always silently discarded on such errors.
- If the "Page Request" needs a response, then the IOMMU itself may generate a "Page Request Group Response" message to the device.

When the IOMMU generates the response, the status field of the response depends on the cause of the error.

The status is set to Response Failure if the following faults are encountered:

- All inbound transactions disallowed (cause = 256)

- DDT entry load access fault (cause = 257)
- DDT entry misconfigured (cause = 259)
- DDT entry not valid (cause = 258)
- Page-request queue is not enabled (`pqcsr.pqen == 0` or `pqcsr.pqon == 0`)
- Page-request queue encountered a memory access fault (`pqcsr.pqmf == 1`)

The status is set to Invalid Request if the following faults are encountered:

- Transaction type disallowed (cause = 260)

The status is set to Success if no other faults were encountered but the "Page Request" could not be queued due to the page-request queue being full (`pqh == pqt - 1`) or had a overflow (`pqcsr.pqof == 1`).

Note

For an SR-IOV device, even though the Page Request Interface is shared between its PFs and VFs, it sends the requesting Function's ID (PF or VF) in the Requester ID field of the Page Request Message, and the requesting Function's ID must be in the Destination Device ID field of the resulting PRG Response Message. So the cause 257 and 260 may occur in a race condition for example between SW and the device. When a Response Failure is returned it implies that something catastrophic has occurred and this response "This response disables the Page Request Interface at the Function". Due to this a small race, it can lead to PRI being disabled for a VFs in the device.

If `dc.PRPR` bit is set, the page request group response must include the PASID TLP prefix if the request had such prefix. If `dc.PRPR` bit is not set, the page request group response must NOT include the PASID TLP prefix even if the request had such prefix, which means that SSV should be set to 0 in the `DTI_ATS_PAGE_RESP`.

Note

For IOMMU generated "Page Request Group Response" messages that have status Invalid Request or Success, the PRG-response-PASID-required (PRPR) bit when set to 1 indicates that the IOMMU response message should include a PASID prefix if the associated "Page Request" had a PASID prefix.
For IOMMU generated "Page Request Group Response" with response code set to Response Failure, if the "Page Request" had a PASID prefix then response is generated with a PASID prefix.

No faults are logged in the fault queue for PCIe ATS "Page Request" messages when:

- Page-request queue is not enabled (`pqcsr.pqen == 0` or `pqcsr.pqon == 0`)
- Page-request queue encountered a memory access fault (`pqcsr.pqmf == 1`)
- Page-request queue full

3.6 PDS Interface Module

AXI4 protocol
Master interface

The PDS interface module implements an arbiter between the various PTWn modules, Queue Manager and the CID Module. It performs read/write requests to system memory to retrieve Page Table Entries or ContextID table entries for instance. This module must always ensure that a 4KiB page is not crossed. A different AXI ID can be used per PTWn, as no ordering is required among the Page Table Walkers.

Error Records: The PDS interface module can detect the following errors:

- SLVERR or DECERR on memory accesses. In this case, the module may raise an access fault.

Per AXI protocol, slaves must always respond, so there is no timeout logic on the master port.

Note

The PDS Master Port can only generate the INCR burst type, and not WRAP or FIXED.

3.7 Queue Manager Module

The Queue Manager Module handles the read and write pointers of the M and S Command queues as well as the M and S Error Record queues.

In IOMMU-22, these queues are memory mapped in system memory. Accesses to the queues happen through the PDS interface. They are implemented as circular buffers and controlled by software via a couple of local registers. Each queue is 4B wide and has a configurable number of slots.

3.7.1 Command Queue

IOMMU-22 holds both the head and tail pointers in registers: `cqh` and `cqt` registers. The head pointer is maintained by the Queue Manager and it points to the first valid command of the queue. The tail pointer, once initialized, is managed by the host software and it points to the last valid command in the queue.

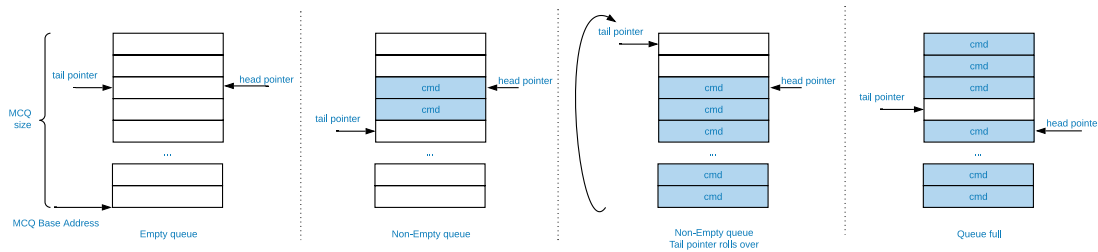


Figure 11: Circular Buffer Command Queue Example

In this example of the Command queue, its size and base address are determined by the `cqb` register.

The tail pointer (`cqt`) is maintained by the host software. Software first ensures that there are enough available slots in the queue before writing one or several commands in the queue. Software then updates the tail pointer by incrementing it with the number of slots it wrote.

The Queue Manager maintains the fill level of the queue by comparing the head and tail pointers. When the queue is not empty, it reads a command from the circular buffer and then updates the head pointer. Software can check the position of the head pointer by reading `qch`.

Software can send many commands in batches to IOMMU-22 via the command queue and IOMMU-22 will execute the commands at its own pace.

IOMMU-22 reads commands from the queue in order but may execute them concurrently. If ordering is required, software must issue a `IOFENCE` command.

`IOFENCE` is a particular command which is handled in the Queue Manager Module. Indeed, the Queue Manager can send commands to various modules in parallel but when an `IOFENCE` command arrives, it stalls the reading of the command queue and wait for all pending command complete signals to come from the various modules.

If Previous-read fencing is requested (by setting `PR` to 1), the command queue manager must guarantee that all previous reads seen by the IOMMU have made it to a global observation point. It achieves this goal by asserting the 'DrainRd' signal on waiting for the `DrainRdAck` response from the IO bridge.

If Previous-write fencing is requested (by setting `PW` to 1), the command queue manager must guarantee that all previous writes seen by the IOMMU have made it to a global observation point. It achieves this goal by asserting the 'DrainWr' signal on waiting for the `DrainWrAck` response from the IO bridge.

The CID Module can execute one command at a time. Once completed, it sends a `CmdComplete` signal back to the Queue Manager which can then send the next command to execute.

The TLB and PTW modules receive the same command to invalidate translation rules. Once completed, each module sends a `CmdComplete` signal back to the Queue Manager. It must get the three `CmdComplete` signals in order to consider it done.

At most, only two commands can be executed in parallel.

Error detection:

- The Queue Manager must check the validity of the command before sending to the right module and thus it can set the illegal command status in `cqcsr`
- The queue being physically instantiated in system memory, the Queue Manager can get an access fault and thus it must set the associated error status in `cqcsr`.

When these errors happen, the queue stalls till the error status bit is cleared. An interrupt is generated if not already pending (i.e. `cip == 1`). To re-enable command processing, software should clear the error status bit by writing 1.

3.7.2 Fault Queue

Errors are reported by IOMMU-22 through the following method:

- An Error Record is logged into the Fault Queue, if the queue has been initialized and if the error is subject to logging.

Similar to the Command queue, the Fault queue is a circular buffer. IOMMU-22 will write Error Records into the queue and the host software will read from the queue whenever the queue is not empty. IOMMU-22 maintains the tail pointer and the host software maintains the head pointer. There is one Fault queue.

Out of reset, the error logging mechanism is disabled.

When the Error Record queue is full, any incoming Error Record from the other modules are dropped.

3.8 Host interface Module

AXI4 protocol
Slave interface

This module contains all IOMMU-22 registers except the Hardware Performance Monitor registers. As the HPM module is optional in IOMMU, the registers are located in the HPM module itself.

This module also manages the INT interrupt line.

The Host interface Module must respond to the entire 4KiB region.

A write to a read-only register is responded with an OKAY response, but the write data is ignored (meaning that the data is not actually written in the register).

Unsupported write or read transactions on populated addresses result in a SLVERR.

3.9 Hardware Performance Monitor Module

See the SiFive IOMMU-22 Manual for details.

Implements 16 counters/events registers (including `iohpmcycles`), implements 40-bit counters.

Chapter 4

Example Device Tree

The device tree will depend on the customer IO-Bridge design.

The following Device Tree excerpt shows a simple IO-Bridge example.

The full file is available here:

https://github.com/avpatel/linux/blob/riscv_iommu_dtbind_v1/Documentation/devicetree/bindings/iommu/riscv%2Ciommu.yaml

```

/* Example 1 (IOMMU platform device with wired interrupts) */
immu1: iommu@1bccd000 {
    compatible = "vendor,chip-iommu", "riscv,iommu";
    reg = <0x1bccd000 0x1000>;
    interrupt-parent = <&aplic_smode>;
    interrupts = <32 4>, <33 4>, <34 4>, <35 4>;
    #iommu-cells = <2>;
};
/* Device with two IOMMU device IDs, 0 and 7 */
master1 {
    iommus = <&immu1 0 1>, <&immu1 7 1>;
};

/* Example 2 (IOMMU platform device with MSIs) */
immu2: iommu@1bccd000 {
    compatible = "vendor,chip-iommu", "riscv,iommu";
    reg = <0x1bccd000 0x1000>;
    msi-parent = <&imsics_smode>;
    #iommu-cells = <2>;
};
bus {
    #address-cells = <2>;
    #size-cells = <2>;
    /* Device with IOMMU device IDs ranging from 32 to 64 */
    master1 {
        iommus = <&immu2 32 32>;
    };
    pcie@400000000 {
        compatible = "pci-host-cam-generic";
        device_type = "pci";
        #address-cells = <3>;
        #size-cells = <2>;
        bus-range = <0x0 0x1>;
        /* CPU_PHYSICAL(2) SIZE(2) */
        reg = <0x0 0x400000000 0x0 0x10000000>;
        /* BUS_ADDRESS(3) CPU_PHYSICAL(2) SIZE(2) */
        ranges = <0x01000000 0x0 0x01000000 0x0 0x01000000 0x0 0x00010000>,
                <0x02000000 0x0 0x41000000 0x0 0x41000000 0x0 0x3f000000>;
        #interrupt-cells = <0x1>;
        /* PCI_DEVICE(3) INT#(1) CONTROLLER(PHANDLE) CONTROLLER_DATA(2) */
        interrupt-map = < 0x0 0x0 0x0 0x1 &aplic_smode 0x4 0x1>,
                        < 0x800 0x0 0x0 0x1 &aplic_smode 0x5 0x1>,
                        <0x1000 0x0 0x0 0x1 &aplic_smode 0x6 0x1>,
                        <0x1800 0x0 0x0 0x1 &aplic_smode 0x7 0x1>;
        /* PCI_DEVICE(3) INT#(1) */
        interrupt-map-mask = <0xf800 0x0 0x0 0x7>;
        msi-parent = <&imsics_smode>;
        /* Devices with bus number 0-127 are mastered via immu2 */
        iommu-map = <0x0000 &immu2 0x0000 0x8000>;
    };
};

```

Note

IOMMU-22 outputs only one interrupt, not 4 as provisioned in RVI in the above example.

Chapter 5

Implementation

This chapter will describe the steps necessary to synthesize the iommu22_sv48_ats_max.

5.1 Top-Level

The top-level Verilog module is defined in: `rtl/design/*_Top.sv`.

All top-level signals are described in Section 1.4.

The following list represents a generalized example flow for use when implementing SiFive cores.

Note

These steps are provided as guidelines only and do not represent a reference flow.

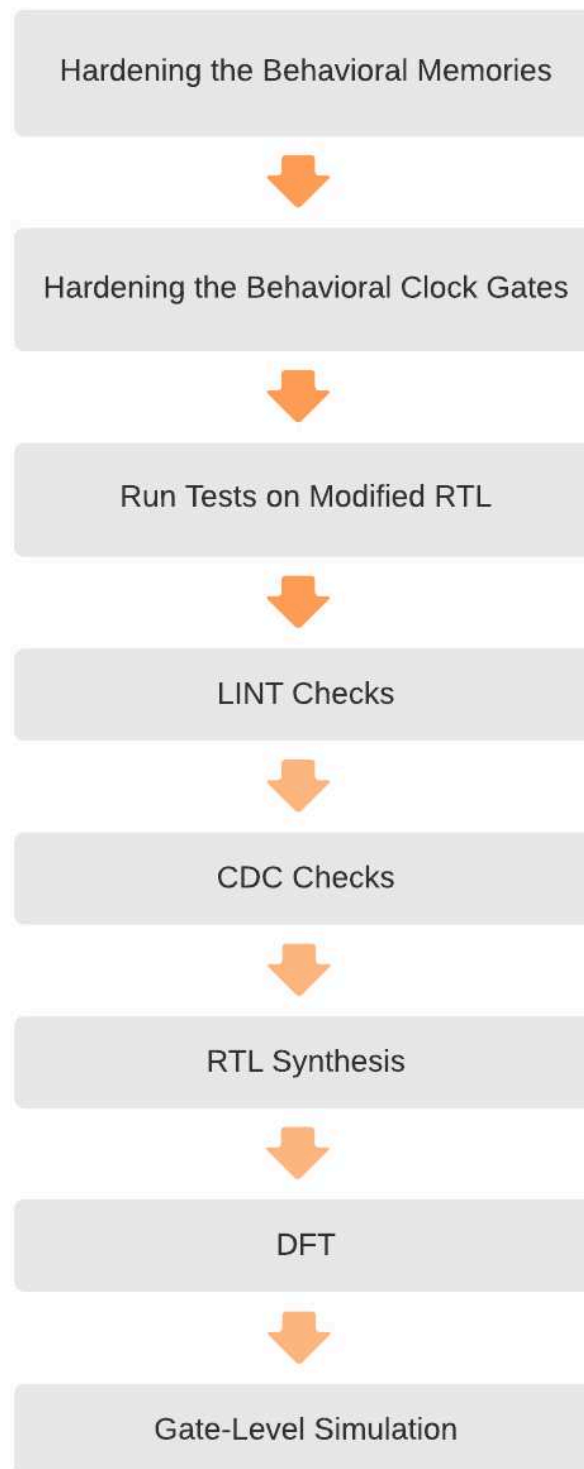


Figure 12: Example Implementation Flow

5.2 Memories

This section describes the memories used in the iommu22_sv48_ats_max design.

5.2.1 RAM Instances

You must ensure that the physical RAMs incorporated in the iommu22_sv48_ats_max substituted for the generic RAMs included in the RAM wrappers satisfy the following requirements:

- Synchronous, single-port RAMs must be used
- All timing must be synchronous to the rising edge of the master clock
- Single-cycle access time at the main clock frequency
- The clock can be held static during inactive cycles, as there is no minimum frequency requirement
- Variable bit-write control is required for some RAM blocks
- Verilog simulation models are required for running the integration tests

Figure 13 shows the timing of the L1 RAM access.

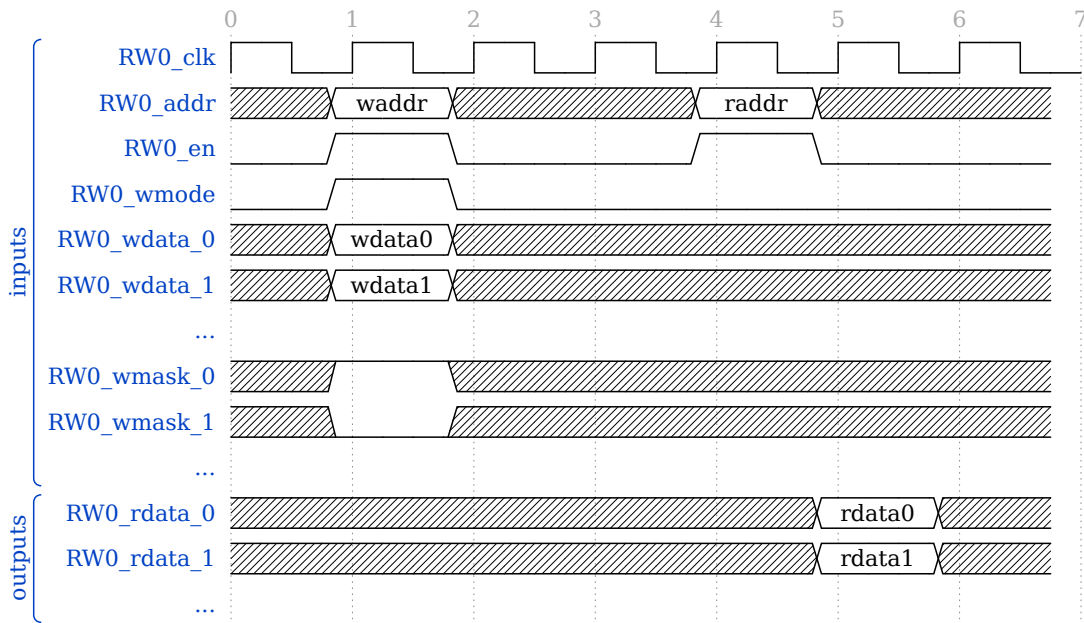


Figure 13: SRAM Timing

The write access occurs during cycle 1, and the read access occurs during cycle 4. Read and write data may be split into smaller chunks. The write mask may also be separated out into bits. In the example write access shown, RW0_wmask_0 is high and RW0_wmask_1 is low. This results in the RW0_wdata_0 segment being written, while the RW0_wdata_1 segment is not.

The IOMMU RAM instances consist of synchronous single-ported SRAMs. These are contained within wrapper modules that expose a standardized generic interface for the iommu22_sv48_ats_max. For each of the modules specified in Table 11, the implementer is required to provide a module definition that instantiates the SRAM macros and connects the macro-specific pins to the interface described in Table 12.

The iommu22_sv48_ats_max RAM instances are delivered as is and are not configurable. It is, however, possible to construct the memory instances from multiple smaller instances. The "Instances" column in the below table describes the distribution of these smaller instances across various components.

The memory instances are also described in the `info/mems.conf` file in the IP deliverables. The "Write Mask Granularity" column in the below tables correspond to `mask_gran` and `ports mrw` in `mems.conf`. The `mask_gran` field is the granularity of the write mask for the logical memory and `mrw` indicates that the logical memory has mask, read, and write ports. For example, a memory of width 156 ("Data Width" column) and `mask_gran` of 39 will have 4 mask bits, each of which controls the write masking of 1/4 of the RAM bits (39 bits).

Module Name	Instances	Depth	Addr. Width (N_{addr})	Data Width (N_{data})	Write Mask Gran. (N_{part})	Description
Iommu22_Sv48MaxAts_l2_dataram_ext	1	512	9	944	118	IOMMU-22 L2 RAM
Iommu22_Sv48MaxAts_ptw_dataram_ext	1	480	9	40		IOMMU-22 Page Table Walker

Table 11: IOMMU SRAM Modules and Configuration

Name	Direction	Width	Description
RW0_clk	Input	1	Memory clock
RW0_en	Input	1	Active-high signal indicating that the memory is being access. This may be used for clock gating.
RW0_addr	Input	N_{addr}	Address of access.
RW0_rdata	Output	N_{data}	Read data
RW0_wmode	Input	1	Active-high signal indicating that the access is a write operation.
RW0_wdata	Input	N_{data}	Write data.
RW0_wmask	Input	N_{data}/N_{part}	Active-high write mask. Each bit controls whether or not the corresponding N_{part} -bit subword is written. This is present only in memories that require mask write functionality.

Table 12: SRAM Signals

5.3 Clock Gates

SiFive cores provide coarse-grained architectural clock gating. This feature enables the core to automatically gate its own clock(s) based on core activity level. Additionally, this feature enables clock gating of other functional units when they enter an inactive state.

The file `rtl/design/EICG_wrapper.sv` defines clock gating logic and is instantiated wherever the clock gates are present.

You must replace the clock gating cell RTL with a clock gating cell from your technology library. This cell must have a test enable input to make the clock gate transparent during scan. For example:

```
module EICG_wrapper(  
    // Outputs  
    out,  
    // Inputs  
    en, test_en, in  
);  
    input en;  
    input test_en;  
    input in;  
    output out;  
  
    <process specific ICG cell> latch(  
        .Q(out),  
        .CP(in),  
        .E(en),  
        .TE(test_en)  
    );  
endmodule
```

5.4 Simulating with Modified RTL

To run simulations with your technology-specific memory instances and clock gating cells, you will need to add their behavioral models to the compilation filelist. The filenames of the memory models can be placed in `rtl/memories.F`, and the filename containing the clock gating cell model can be placed in `rtl/design.F`.

5.5 LINT

Linting simulations are performed using the Synopsys® SpyGlass® platform.

Note

Simulation runs should produce zero errors and thus waiver files are not included in the product deliverables.

The SpyGlass lint goal groups are shown below:

```
LINT:
  lint/lint_rtl
  lint/lint_turbo_rtl
```

5.5.1 Directory Structure

SpyGlass simulations use the following file types:

- *.f - List of all Verilog design files plus auxiliary Verilog files
- *.prj - Project file that orchestrates spyglass simulation
- *.sgdc - File that specifies top level constraints for the design
- *.sh - Rerun file for the spyglass simulation

These lint-related files reside at the following locations within the directory structure:

```
-- Makefile
-- rtl
|  |-- design
|  |-- implementation
|  |  |-- lint
|  |  |  |-- LINT.f
|  |  |  |-- LINT.prj
|  |  |  |-- LINT.sgdc
|  |  |  |-- LINT_rerun.sh
```

5.5.2 Running Lint Simulations

The following make targets are provided to facilitate lint simulation:

- make spyglass_all - performs simulation for the following goals:

```
lint/lint_rtl
lint/lint_turbo_rtl
```

- make spyglass_lint - performs simulation for the following goals:

```
lint/lint_rtl
lint/lint_turbo_rtl
```

The following result directories will be created for Lint simulation runs:

```
-- Makefile
-- rtl
|  |-- implementation
|  |  |-- lint
|  |  |  |-- <core_prefix>_LINT
```


5.6 CDC

Simulation of CDC-related components is performed using Synopsys SpyGlass.

Note

Simulation runs should produce zero errors and thus waiver files are not included in the product deliverables.

The SpyGlass CDC goals are shown below:

```
CDC:
  cdc/cdc_verify_struct
  cdc/clock_reset_integrity
```

5.6.1 Directory Structure

Users may perform simulations using four relevant file types:

- *.f - List of all Verilog design files plus auxiliary Verilog files
- *.prj - Project file that orchestrates spyglass simulation
- *.sgdc - File that specifies top level constraints for the design
- *.sh - Rerun file for the spyglass simulation

These CDC-related files reside at the following locations within the directory structure:

```
|-- Makefile
|-- rtl
|   |-- design
|   |-- implementation
|   |   |-- cdc
|   |       |-- CDC.f
|   |       |-- CDC.prj
|   |       |-- CDC.sgdc
|   |       |-- CDC_rerun.sh
```

5.6.2 Running CDC Simulations

The following make targets are provided to facilitate simulation:

- make spyglass_all - performs simulation for the following CDC goals:

```
cdc/cdc_verify_struct
cdc/clock_reset_integrity
```

- make spyglass_cdc - performs simulation for the following CDC goals:

```
cdc/cdc_verify_struct  
cdc/clock_reset_integrity
```

The following result directories will be created for CDC simulation runs:

```
|-- Makefile  
|-- rtl  
|   |-- implementation  
|   |   |-- cdc  
|   |   |-- <core_prefix>_CDC
```

5.7 Synthesis

No defines are required to run synthesis.

5.7.1 SDC File

The IP delivery comes with an SDC file to help you get started with correct timing constraints. The main SDC file is located at `/rtl/implementation/synth/constraints.sdc`.

This SDC contains the following constraints:

- Creation of all clocks with variables available to override your needed period
- Clock group creation to ensure synchronous or asynchronous relationships between clocks
- Input and output delays against the correct clock for every I/O

For all SDC files, it is expected that you will create the following constraints as needed. This is not intended to be an exhaustive list of constraints you may need:

- Output load
- Input driving cell
- Uncertainty
- Design rule constraints (max transition, max capacitance, max fanout)
- Asynchronous CDC constraints (if needed for the design)

For most SiFive IP, the following constraints are not needed:

- False paths
- Multi-cycle paths, even between clock domains

However, for some configuration options, the design may include false paths, and/or multi-cycle paths. In such cases, the provided SDC file will include all the necessary constraints for such paths. If the SDC file does not include any such constraints, it will imply they are not needed for the particular design.

As provided, the SDC file should be compatible with any tool that supports the SDC file format.

5.7.2 Retiming

A list of all modules (if any) that require retiming is included in the design deliverables. The list can be found in the `rtl/implementation/synth/modules_to_be_retimed.json` file.

Ensure that those modules are marked for register retiming when running synthesis. Otherwise, they won't be able to achieve as high a frequency if retiming is not applied. During synthesis, please specify the right set of commands that will instruct the synthesis tool to perform retiming on these modules. Please consult with your EDA tool vendor to use the right command and arguments to specify this.

5.8 DFT

For a flip-flop to be included in a scan chain, its clock input must be directly controllable by a design input. Therefore, the following input signal is used during post-silicon test to make clock gaters transparent.

DFT Input Signal Name	Description
<code>psd_test_clock_enable</code>	When <code>psd_test_clock_enable= 1</code> , the clock gaters are transparent. All the flip-flops' clock input are directly connected to a primary clock input of the design.

Table 13: DFT Input Signals

5.8.1 DFT Rule Check

Simulation of DFT-related components is performed using the Synopsys SpyGlass platform.

Note

Simulation runs should produce zero errors, and thus waiver files are not included in the product deliverables.

The SpyGlass DFT goal groups are shown below:

DFT:
dft/dft_scan_ready

5.8.2 Directory Structure

SpyGlass simulations use the following file types:

- *.f - List of all Verilog design files plus auxiliary Verilog files

- *.prj - Project file that orchestrates spyglass simulation
- *.sgdc - File that specifies top level constraints for the design
- *.sh - Rerun file for the spyglass simulation

These DFT-related files reside at the following locations within the directory structure:

```
|-- Makefile
|-- rtl
|   |-- design
|   |-- implementation
|   |   |-- dft
|   |       |-- DFT.f
|   |       |-- DFT.prj
|   |       |-- DFT.sgdc
|   |       |-- DFT_rerun.sh
```

5.8.3 Running DFT Rule Checks

The following make targets are provided to facilitate DFT simulation:

- make spyglass_all - performs simulation for the following DFT goal: dft/dft_scan_ready
- make spyglass_dft - performs simulation for the following DFT goal: dft/dft_scan_ready

The following result directories will be created for DFT simulation runs:

```
|-- Makefile
|-- rtl
|   |-- implementation
|   |   |-- dft
|   |       |-- <core_prefix>_DFT
```

5.9 Gate-Level Simulation

To avoid X-propagation, all state such as flip-flops and SRAMs must be initialized at the beginning of simulation, as well as after each power-up event if conducting power-aware simulation. Additionally, ensure that the SRAM models never produce X values by modifying them as necessary.

One possible initialization method is to use the Verilog simulator's +initreg and +initmem option. You may need to limit the scope of this initialization to only include the SiFive design.

Appendix A

Configuration Options

This section provides a reference of the three supported variants of IOMMU-22. The file `docs/systemip_configuration.txt` lists the features and options configured in the `iommu22_sv48_ats_max`.

Variant:

- Three selectable variants (Small, Medium, Large) with progressively larger area
 - Small - For embedded system with Context information contained within the IOMMU
 - Medium - For systems with large number of devices or for high-throughput masters, such as PCIe without PASID support
 - Large - For high-throughput masters such as PCIe Gen 5 with PASID support

Note that the following settings are hard-wired per variant and are not individually configurable outside of the selected variant.

Device ID:

- Small - 8-bit
- Medium - 16-bit
- Large - 16-bit

Process ID:

- Small - Not present
- Medium - Not present
- Large - 20-bit

ATS Support:

- Small - Not present
- Medium - Present
- Large - Present

TLB L1 and L2:

- Small - L1 = 64-Entry, L2 = 1024-Entry
- Medium - L1 = 64-Entry, L2 = 2048-Entry
- Large - L1 = 64-Entry, L2 = 4096-Entry

PTE Cache and PTW:

- Small - 16/32/64/64 PTE cache, 4 Parallel PTW
- Medium - 32/64/128/256 PTE cache, 8 Parallel PTW
- Large - 32/64/128/256 PTE cache, 8 Parallel PTW

Device Directory Table and ProcessID Directory Table:

- Small - 64-Entry Cache, ProcessID Directory Table cache not present
- Medium - 128-Entry Cache, ProcessID Directory Table cache not present
- Large - 128-Entry Cache, 128-Entry ProcessID Directory Table cache

Appendix B

Top-Level Signals

This section provides a reference for the top-level signals in the iommu22_sv48_ats_max. Note that the actual signals in your design may vary from the list below depending on your iommu-22 variant. The complete list of top-level signals is available in the file `rtl/design/*_Top.sv` of your IP deliverables package.

The top-level signals are described in more detail in Section 1.4.

```

input 1 clock
input 1 reset
input 1 psd_test_clock_enable
input 1 pds_axi4_0_aw_ready
input 1 pds_axi4_0_w_ready
input 1 pds_axi4_0_b_valid
input 4 pds_axi4_0_b_bits_id
input 2 pds_axi4_0_b_bits_resp
input 1 pds_axi4_0_ar_ready
input 1 pds_axi4_0_r_valid
input 4 pds_axi4_0_r_bits_id
input 64 pds_axi4_0_r_bits_data
input 2 pds_axi4_0_r_bits_resp
input 1 pds_axi4_0_r_bits_last
input 1 host_ahb_0_hmastlock
input 1 host_ahb_0_hsel
input 1 host_ahb_0_hready
input 2 host_ahb_0_htrans
input 3 host_ahb_0_hsize
input 1 host_ahb_0_hburst
input 1 host_ahb_0_hwrite
input 4 host_ahb_0_hprot
input 13 host_ahb_0_haddr
input 64 host_ahb_0_hwdata
input 1 dbg_disable
input 1 iommu_mode
input 1 eb_reset
input 1 source_cxl
input 1 dtri_valid
input 20 dtri_bits_pid
input 16 dtri_bits_did
input 1 dtri_bits_pid_valid
input 38 dtri_bits_va
input 8 dtri_bits_reqid
input 1 dtri_bits_rwn
input 1 dtri_bits_translated
input 1 dtri_bits_exe
input 1 dtri_bits_priv
input 1 dtci_ready
input 1 drain_rd_ack
input 1 drain_wr_ack
input 1 pma_lookup_req_ready
input 1 pma_lookup_resp_valid
input 2 pma_lookup_resp_bits_pma
input 1 dtis_dn_valid
input 256 dtis_dn_bits_data
input 32 dtis_dn_bits_keep
input 1 dtis_dn_bits_dest
input 1 dtis_dn_bits_id
input 1 dtim_up_ready
output 1 pds_axi4_0_aw_valid
output 4 pds_axi4_0_aw_bits_id
output 48 pds_axi4_0_aw_bits_addr
output 8 pds_axi4_0_aw_bits_len
output 3 pds_axi4_0_aw_bits_size
output 2 pds_axi4_0_aw_bits_burst
output 1 pds_axi4_0_aw_bits_lock
output 4 pds_axi4_0_aw_bits_cache

```



```
output 3   pds_axi4_0_aw_bits_prot
output 4   pds_axi4_0_aw_bits_qos
output 1   pds_axi4_0_w_valid
output 64  pds_axi4_0_w_bits_data
output 8   pds_axi4_0_w_bits_strb
output 1   pds_axi4_0_w_bits_last
output 1   pds_axi4_0_b_ready
output 1   pds_axi4_0_ar_valid
output 4   pds_axi4_0_ar_bits_id
output 48  pds_axi4_0_ar_bits_addr
output 8   pds_axi4_0_ar_bits_len
output 3   pds_axi4_0_ar_bits_size
output 2   pds_axi4_0_ar_bits_burst
output 1   pds_axi4_0_ar_bits_lock
output 4   pds_axi4_0_ar_bits_cache
output 3   pds_axi4_0_ar_bits_prot
output 4   pds_axi4_0_ar_bits_qos
output 1   pds_axi4_0_r_ready
output 1   host_ahb_0_hreadyout
output 1   host_ahb_0_hresp
output 64  host_ahb_0_hrdata
output 1   wint
output 1   eb_error_id
output 1   dtci_ready
output 1   dtci_valid
output 36  dtci_bits_pa
output 2   dtci_bits_pbmt
output 8   dtci_bits_reqid
output 1   dtci_bits_error_flag
output 1   drain_rd
output 1   drain_wr
output 1   pma_lookup_req_valid
output 48  pma_lookup_req_bits_pa
output 4   pma_lookup_req_bits_size
output 1   pma_lookup_resp_ready
output 1   dtis_dn_ready
output 1   dtim_up_valid
output 256 dtim_up_bits_data
output 32  dtim_up_bits_keep
output 1   dtim_up_bits_dest
output 1   dtim_up_bits_id
```

Appendix C

Revision History

Version	Date	Document Changes
24G1.02.00	April 5, 2024	• <i>Appendix A</i>: Added Configuration Options appendix • <i>Appendix B</i>: Added Top-Level Signals appendix • Moved Programming Model chapter to Manual • Editorial fixes
24G1.01.00	February 19, 2024	• Initial release

Table 14: Revision Information